

Universal MCP Server — Schofield Structured File Organization

A production-ready, extensible Python project structure for building a **universal Model Context Protocol (MCP) server** that connects AI hosts to any external capability via Tools, Resources, and Prompts.

📁 Directory Tree

```
universal-mcp/
├── pyproject.toml          # Project metadata & dependencies
├── README.md              # Project documentation
├── .env.example           # Environment variable template
├── Dockerfile             # Container packaging
├── docker-compose.yml     # Multi-service orchestration
├── mcp_config.json        # Host configuration template
├──
├── src/
│   └── universal_mcp/
│       ├── __init__.py    # Package version & exports
│       ├── __main__.py    # CLI entry point: `python -m universal_mcp`
│       ├──
│       ├── server.py      # 🧠 Core MCP server orchestrator
│       │                  #   - Registers all primitives
│       │                  #   - Manages lifecycle (init → run → shutdown)
│       │                  #   - Bridges transport primitives
│       ├──
│       ├── config.py      # ⚙️ Centralized configuration
│       │                  #   - Pydantic settings from env/JSON/YAML
│       │                  #   - Transport selection, auth, sandbox paths
│       ├──
│       ├── transports/
│       │   ├── __init__.py # Transport factory & exports
│       │   ├── base.py     # Abstract Transport interface
│       │   ├── stdio.py    # 🖥️ STDIO transport (local desktop hosts)
│       │   └── http_sse.py # 🌐 HTTP/SSE transport (remote/network hosts)
│       ├──
│       ├── primitives/    # MCP's three core primitives
│       │   ├── __init__.py
│       │   ├──
│       │   ├── tools/     # 🛠️ TOOLS = Actions the AI can execute
│       │   │   ├── __init__.py
│       │   │   ├── registry.py # Tool discovery, registration, listing
│       │   │   ├── base.py    # Abstract Tool class with schema validation
│       │   │   └── builtin/   # Built-in universal tools (ships with server)
│       │   │       ├── __init__.py
│       │   │       ├── filesystem.py # read_file, write_file, list_dir, search
│       │   │       ├── system.py    # execute_command, get_env, get_processes
│       │   │       ├── web.py       # fetch_url, search_web, parse_html
│       │   │       ├── code.py      # run_python, lint_code, format_code
│       │   │       └── database.py  # query_sql, migrate, backup
```

```

|   |   |   |
|   |   |   |   | resources/          # 📁 RESOURCES = Data/context the AI can read
|   |   |   |   |   | __init__.py
|   |   |   |   |   | registry.py      # Resource URI registration & discovery
|   |   |   |   |   | base.py          # Abstract Resource class
|   |   |   |   |   |   | builtin/      # Built-in universal resources
|   |   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   |   | file_content.py # file://path/to/file
|   |   |   |   |   |   |   | env_vars.py    # env://VAR_NAME
|   |   |   |   |   |   |   | system_info.py  # system://cpu|memory|disk
|   |   |   |   |   |   |   | logs.py        # log://app/2024-01-01
|   |   |   |   |   |
|   |   |   |   |   | prompts/          # 📝 PROMPTS = Reusable instruction templates
|   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   | registry.py      # Prompt template registration
|   |   |   |   |   |   | base.py          # Abstract Prompt class with arg substitution
|   |   |   |   |   |   |   | builtin/      # Built-in universal prompts
|   |   |   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   |   |   | code_review.py    # "Review this code for bugs..."
|   |   |   |   |   |   |   |   | debug_analysis.py # "Analyze this stack trace..."
|   |   |   |   |   |   |   |   | project_setup.py  # "Initialize a Python project..."
|   |   |   |   |   |   |   |   | doc_generate.py   # "Generate API docs from..."
|   |   |   |   |   |
|   |   |   |   |   | security/          # 🛡️ Security & sandboxing
|   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   | auth.py         # OAuth2, API key, Bearer token handlers
|   |   |   |   |   |   | rate_limiter.py  # Token bucket / sliding window rate limits
|   |   |   |   |   |   | sandbox.py       # Path restriction, command allow-listing
|   |   |   |   |   |   | audit.py        # Request/response logging for compliance
|   |   |   |   |   |
|   |   |   |   |   | utils/
|   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   | logging.py      # 📁 File-based logging (NEVER stdout in STDIO mode)
|   |   |   |   |   |   | validators.py    # Input sanitization & schema validation
|   |   |   |   |   |   | errors.py        # Custom exception hierarchy
|   |   |   |   |   |   | helpers.py       # Common utilities (async wrappers, caching)
|   |   |   |   |   |
|   |   |   |   |   | connectors/         # 🚀 External service adapters
|   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   | base.py          # Abstract connector interface
|   |   |   |   |   |   | database.py      # PostgreSQL, MySQL, SQLite adapters
|   |   |   |   |   |   | api_client.py    # Generic REST/GraphQL client
|   |   |   |   |   |   | ssh.py           # Remote SSH command execution
|   |   |   |   |   |   | docker.py        # Docker container management
|   |   |   |   |   |
|   |   |   |   |   | tests/
|   |   |   |   |   |   | __init__.py
|   |   |   |   |   |   | conftest.py     # Pytest fixtures & shared mocks
|   |   |   |   |   |   | test_server.py  # Server lifecycle & integration tests
|   |   |   |   |   |   | test_tools.py   # Tool execution & schema tests
|   |   |   |   |   |   | test_resources.py # Resource URI resolution tests

```

```

|   ├── test_prompts.py           # Prompt template rendering tests
|   ├── test_transports.py        # STDIO & HTTP transport tests
|   └── test_security.py          # Auth, sandbox, rate-limit tests
|
|  └── docs/
|      ├── architecture.md        # Mental model & design decisions
|      ├── tools_reference.md      # Auto-generated tool catalog
|      ├── resources_reference.md  # Auto-generated resource catalog
|      ├── prompts_reference.md    # Auto-generated prompt catalog
|      └── deployment.md          # Docker, K8s, cloud deployment guide
|
└── scripts/
    ├── install.sh                # One-line install for end users
    ├── dev.sh                    # `mcp dev` wrapper with hot reload
    ├── test.sh                   # Run full test suite with coverage
    └── lint.sh                   # ruff + mypy + bandit checks

```

Component Mapping to MCP Architecture

MCP Concept	File Location	Role
Host	External (Claude Desktop, Cursor, VS Code)	AI application that consumes the server
Client	External (built into host)	Protocol handler; <i>you do NOT build this</i>
Server	src/universal_mcp/server.py	Service provider you build
Tools	src/universal_mcp/primitives/tools/	Hands —actions the AI can execute
Resources	src/universal_mcp/primitives/resources/	Knowledge —data the AI can access
Prompts	src/universal_mcp/primitives/prompts/	Templates —reusable instruction workflows
Transport	src/universal_mcp/transports/	Network —STDIO (local) or HTTP/SSE (remote)

Quick Start

1. Installation

```

# Clone the repository
git clone https://github.com/your-org/universal-mcp.git
cd universal-mcp

# Install with uv (recommended)

```

```
uv pip install -e ".[dev]"

# Or with pip
pip install -e ".[dev]"
```

2. Configure Environment

```
cp .env.example .env
# Edit .env to set transport, auth, and sandbox paths
```

3. Run the Server

```
# STDIO mode (for Claude Desktop, Cursor)
python -m universal_mcp --transport stdio

# HTTP/SSE mode (for remote/network access)
python -m universal_mcp --transport http --port 3000

# Development mode with MCP Inspector
mcp dev src/universal_mcp/server.py
```

4. Connect Your Host

Add to your host's MCP configuration:

Claude Desktop (~/.Library/Application Support/Claude/claude_desktop_config.json):

```
{
  "mcpServers": {
    "universal": {
      "command": "python",
      "args": ["-m", "universal_mcp", "--transport", "stdio"],
      "env": {
        "MCP_SANDBOX_PATH": "/allowed/path",
        "MCP_LOG_LEVEL": "INFO"
      }
    }
  }
}
```

Cursor (.cursor/mcp.json):

```
{
  "mcpServers": {
    "universal": {
      "command": "python",
      "args": ["-m", "universal_mcp", "--transport", "stdio"]
    }
  }
}
```

Key Files Explained

src/universal_mcp/server.py

The central orchestrator. Initializes the MCP server, registers all primitives (tools, resources, prompts), and binds to the selected transport.

src/universal_mcp/config.py

Pydantic-based configuration. Supports environment variables, .env files, and JSON/YAML config files. Validates sandbox paths, auth tokens, and transport settings at startup.

src/universal_mcp/transports/

- **STDIO**: For local desktop hosts. Uses stdin/stdout for JSON-RPC 2.0 messages. *Critical*: logging must go to files, never stdout.
- **HTTP/SSE**: For remote/network hosts. Supports Server-Sent Events for streaming and standard HTTP auth (Bearer, API key, OAuth2).

src/universal_mcp/primitives/tools/

Each tool is a class inheriting from `BaseTool` with:

- **name**: snake_case identifier (e.g., `read_file`, `execute_command`)
- **description**: LLM-friendly explanation
- **input_schema**: JSON Schema for parameter validation
- **execute()**: Async method implementing the action

src/universal_mcp/primitives/resources/

Each resource is addressable by URI:

- `file:///path/to/file.txt`
- `env://HOME`
- `system://memory`
- `log://app/2024-06-01`

src/universal_mcp/primitives/prompts/

Reusable templates with Jinja2-style argument substitution:

- `code_review`: “Review the following `{{language}}` code for security issues...”
- `debug_analysis`: “Analyze this stack trace from `{{framework}}`...”

src/universal_mcp/security/

- **Auth**: OAuth2, API key, Bearer token validation
- **Rate Limiter**: Token bucket per client/session
- **Sandbox**: Path restriction, command allow-listing, file size limits
- **Audit**: Immutable request/response logs for compliance

src/universal_mcp/utils/logging.py

File-based logging to `/var/log/universal_mcp/` or user-specified path. *Never* writes to stdout in STDIO transport mode to avoid corrupting JSON-RPC messages.

Security Best Practices

1. **Sandbox all filesystem operations** — restrict to allowed paths only
 2. **Allow-list shell commands** — never pass raw user input to subprocess
 3. **Rate limit tool calls** — prevent abuse and resource exhaustion
 4. **Audit everything** — log all tool invocations with timestamps and client IDs
 5. **Validate inputs** — strict JSON Schema validation before execution
 6. **Never log secrets** — redact API keys, tokens, passwords from logs
 7. **Use Docker** — containerize for isolation and reproducibility
-

Docker Deployment

```
# Build
docker build -t universal-mcp:latest .

# Run STDIO mode (for local hosts via docker exec)
docker run -i --rm universal-mcp:latest

# Run HTTP mode
docker run -p 3000:3000 --env-file .env universal-mcp:latest --transport http
```

MCP Server Checklist

- Host selected (Claude Desktop, Cursor, VS Code, custom)
 - Server capabilities defined (Tools / Resources / Prompts)
 - Server code written (MCP Python SDK v1.26+)
 - Transport configured (STDIO or HTTP/SSE)
 - Host configured to connect (`mcp_config.json`)
 - End-to-end test passed (via MCP Inspector)
 - Security implemented (auth, sandbox, rate limiting)
 - Logging to files (not stdout in STDIO mode)
 - Docker packaging complete
 - Documentation generated
-

References

- [Model Context Protocol Specification](#)
- [MCP Python SDK v1.26](#)
- [MCP Architecture Overview](#)

- [Awesome MCP Best Practices](#)

Generated for the Universal MCP Python project — a single server connecting AI to everything.